



Kanea

AI-Powered Customer Support System for ECG

Electricity Company of Ghana — Phase 2 Project Report

Project Name: Kanea AI Customer Support Chatbot

Client Organisation: Electricity Company of Ghana (ECG)

Report Type: Technical Project Report

Date: June 2026

Version: 2.0.0

Status: Development Complete

Table of Contents

1. Executive Summary
2. Background and Problem Statement
3. Project Objectives
4. System Architecture
5. Technology Stack
6. Implementation — Phase by Phase
7. Backend Deep-Dive
8. Frontend Deep-Dive
9. Security Measures

10. Changes and Improvements Made
11. How to Run the System
12. Known Limitations
13. Recommended Future Improvements
14. Conclusion

1. Executive Summary

Kanea is a fully functional AI-powered customer support chatbot designed for the **Electricity Company of Ghana (ECG)**. It provides 24/7 intelligent support to ECG customers through a modern web interface, covering the four most common customer service needs: real-time outage status enquiries, electrical fault reporting, billing and tariff information, and escalation to human agents when necessary.

The system is built as a two-tier web application: a **Python/FastAPI** backend hosting a REST and WebSocket API, and a **React/Vite** frontend delivering a responsive single-page application. The AI engine uses **Groq's free-tier API** (Llama 3.3 70B) combined with a custom Retrieval-Augmented Generation (RAG) pipeline trained on 14 sections of ECG-specific knowledge. Live outage data is scraped directly from *ecg.com.gh* at startup and refreshed every hour automatically.

Key Capabilities at a Glance

- Real-time outage status sourced directly from the live ECG website (8 regional pages scraped)
- Intelligent 3-step query routing: static FAQs vs. live outage data vs. human escalation
- Secure admin dashboard with analytics, fault report queue, and escalation tracking
- Deployable on a local network — accessible from any device on the same Wi-Fi
- Production-grade security: token-based admin auth, rate limiting, XSS protection

2. Background and Problem Statement

ECG serves millions of electricity customers across Ghana's southern regions. Customer service is currently handled primarily through a telephone hotline (**0302 611 611**) and an emergency line (**0800-POWER / 0800-76937**). This creates several well-documented challenges:

- High call volumes during outage events overwhelm the call centre
- Customers cannot self-serve for routine queries such as billing, prepaid tokens, and connection procedures
- Outage status information on the ECG website is not surfaced in a conversational or structured way
- No digital channel exists for logging and tracking electrical fault reports
- After-hours support is limited to the emergency line only

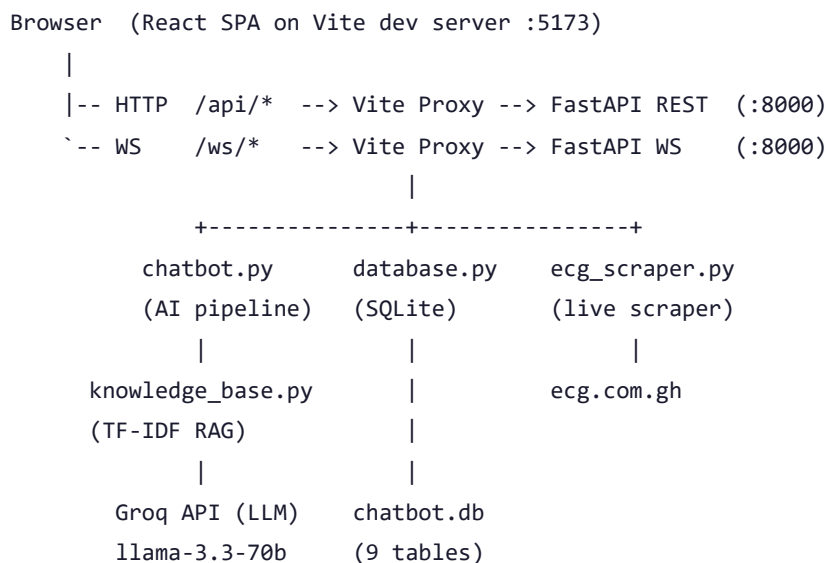
This project was commissioned as **Phase 2** of an ongoing initiative to modernise ECG's customer interaction channels. Phase 1 delivered the conceptual design and presentation; Phase 2 delivers a working, deployed system with live data integration, a complete admin dashboard, and measurable quality metrics.

3. Project Objectives

#	Objective	Status
1	Build a functional AI chatbot that answers ECG customer queries	Complete
2	Integrate live outage data from ecg.com.gh into the chatbot	Complete
3	Provide a fault reporting form with unique reference number tracking	Complete
4	Build an admin dashboard for operations visibility	Complete
5	Measure chatbot precision and user satisfaction	Complete
6	Secure the system against common web vulnerabilities	Complete
7	Make the system accessible across devices on a local network	Complete
8	Deploy to cloud with HTTPS for public access	Future Work

4. System Architecture

Kanea follows a standard **client-server architecture** with a clear separation between the frontend (React SPA) and the backend (FastAPI service). Communication uses two protocols: standard HTTP REST for data queries and form submissions, and WebSocket for the real-time chat interface.



Data Flow — Chat Request

1. User types a message → frontend sends it over the open WebSocket connection
2. Backend saves the message to the `messages` table in SQLite
3. **chatbot.py** classifies the intent (7 categories) and routes the query:
 - *Dynamic* → fetches live outage records from the ECG outage table
 - *Static* → retrieves the top-5 most relevant knowledge base documents via TF-IDF
 - *Escalation* → triggers human agent handoff flag and logs to escalations table
4. A structured system prompt (outage data + RAG context + routing rules) is sent to the Groq LLM
5. The LLM response is cleaned, saved to the database, and returned over WebSocket with intent metadata
6. The frontend renders the message, attaches feedback buttons, and may show a star rating prompt

5. Technology Stack

Backend

Package	Version	Role
<code>fastapi</code>	0.115.0	Web framework — REST API and WebSocket endpoints
<code>uvicorn[standard]</code>	0.30.6	ASGI server with WebSocket support
<code>httpx</code>	0.27.2	Async HTTP client for Groq API calls and web scraping
<code>pydantic</code>	2.9.2	Request/response validation and data modelling
<code>python-dotenv</code>	1.0.1	Loading secrets from the <code>.env</code> file
<code>websockets</code>	13.1	WebSocket protocol support for uvicorn
<code>beautifulsoup4</code>	4.12.3	HTML parsing for the ECG website scraper
<code>slowapi</code>	0.1.9	Rate limiting middleware for FastAPI

Frontend

Library	Role
React 18	Component-based UI framework
Vite	Fast dev server with proxy support; production bundler
Tailwind CSS	Utility-first CSS framework — all styling done inline in JSX
DOMPurify	HTML sanitisation library to prevent XSS from bot responses

External Services

Service	Purpose
Groq API (free tier)	LLM inference — <code>llama-3.3-70b-versatile</code> model
ecg.com.gh	Source of live outage and maintenance notices (scraped)
SQLite	Local relational database — zero-configuration, file-based (<code>chatbot.db</code>)

6. Implementation — Phase by Phase

1 Foundation — Core Chatbot

Built the initial FastAPI backend with WebSocket chat, SQLite database, basic intent classification (keyword scoring), and the React frontend with a chat UI. Integrated the Groq LLM API with a base system prompt. Established the project folder structure:

`backend/`, `frontend/`, `.venv/`.

2 Knowledge Base — RAG Pipeline

Created `knowledge_base.py` to load the ECG presentation slides as a searchable document corpus. Implemented keyword-overlap scoring so the chatbot could retrieve relevant background information before calling the LLM. Added

`presentation_text.txt` with slide content as the first knowledge source.

3 Live Data Integration — ECG Website Scraper

Wrote `ecg_scraper.py` to scrape the ECG website for live outage and maintenance notices. Discovered the correct Joomla CMS URL pattern (`/index.php/en/regional-status/...`) by inspecting the ECG homepage link structure. Created scrapers for the homepage, main status page, and all 8 regional pages (Accra East/West, Tema, Ashanti, Eastern, Western, Volta, Central). Added `ecg_knowledge.txt` with 14 sections of static ECG knowledge (billing, safety, new connections, prepaid meters, etc.) extracted from the ECG website.

4 Decision Logic — Static vs. Dynamic Routing

Upgraded the chatbot engine with a formal 3-step pipeline. Step 1 classifies intent across 7 categories. Step 2 routes to one of three data sources: the live outage database (dynamic), the RAG knowledge base (static), or human escalation. Step 3 builds a tailored system prompt specific to the routing decision and calls the Groq LLM. This architecture prevents the chatbot from inventing outage data for static queries, and from citing irrelevant FAQs for live outage questions.

5 Admin Dashboard and Analytics

Built the AdminPanel React component with 4 tabs: Analytics (KPI cards, charts, recent feedback), Fault Reports queue, Escalations log, and RAG Sources audit trail.

Implemented `Analytics.jsx` with a satisfaction histogram, intent distribution bar chart, and per-message thumbs-up/down breakdown. Added database tables for `chat_ratings`, `message_feedback`, `rag_references`, and `ecg_scrape_log`.

6 Security Hardening and Network Access

Addressed 9 security and quality gaps identified in a systematic self-audit (detailed in Section 10). Key changes: backend token authentication replaced the hardcoded frontend password; `slowapi` rate limiting was added to all REST endpoints; DOMPurify XSS sanitisation was added to the chat UI; both servers were configured to bind to `0.0.0.0` for LAN access; a `.gitignore` file was created to protect secrets and the database file from being committed to source control.

7. Backend Deep-Dive

7.1 AI Pipeline (`chatbot.py`)

Step 1 — Intent Classification

A keyword-scoring NLP pre-classifier (no external NLP library required) scores the user's message against 7 intent categories. Each matching keyword adds one point; the highest-scoring category wins. If no keywords match, the intent defaults to `general`.

Intent	Example Trigger Keywords	Typical Action
<code>outage_status</code>	outage, blackout, dumsor, restore, no light	Fetch live outage data
<code>fault_report</code>	broken pole, fallen wire, spark, transformer	Guide to fault form
<code>billing</code>	bill, prepaid, token, tariff, recharge, vend	RAG knowledge base
<code>safety</code>	shock, surge, hazard, precaution, is it safe	RAG + danger check
<code>escalation</code>	human, agent, supervisor, speak to, urgent	Route to human agent
<code>complaint</code>	terrible, frustrated, always, never, disgusted	RAG + escalation offer
<code>general</code>	(no keyword match)	RAG knowledge base

Step 2 — Query Type Routing

The intent is refined into one of three routing decisions. A dedicated set of dynamic phrases (outage, blackout, restoration, dumsor, load shedding, current status, etc.) can override the intent to force a live data lookup even when intent classification is ambiguous. Danger keywords (fallen wire, electric shock, fire, burning, electrocution) trigger an immediate escalation flag regardless of intent.

Step 3 — LLM Response Generation

A structured system prompt is assembled with: the routing decision and its specific instructions; the full live outage list from the database; and the top-N knowledge base documents (top-5 for static queries, top-2 as supplemental context for dynamic queries). The last 4 conversation turns (8 messages) are included as history for context continuity. The Groq API is called asynchronously with temperature 0.4 and a 600-token output limit.

Session Persistence

Conversation history is stored in-memory during an active session but also written to the `messages` SQLite table after each turn. If the backend server restarts, the chatbot reloads the last 4 turns from the database on the next message, so returning users do not lose their context.

7.2 RAG Knowledge Base (`knowledge_base.py`)

The knowledge base loads documents from two sources at startup and pre-computes TF-IDF weights across the entire corpus. When a query arrives, matching terms are scored by `TF × IDF` per document. The IDF formula is:

$$IDF = \log((N + 1) / (df + 1)) + 1.0$$

where N is the total number of documents and df is the number of documents containing the term. This smoothing ensures rare domain-specific terms (PURC, GRIDCO, prepaid token vending, reconnection fee) score significantly higher than common words.

Source File	Sections	Content
<code>presentation_text.txt</code>	Slide sections (prefix: <code>slide_</code>)	Project background and system overview from Phase 1 slides
<code>ecg_knowledge.txt</code>	14 ECG sections (prefix: <code>ecg_</code>)	Company overview, billing, prepaid meters, new connections, disconnection/reconnection, safety guidelines, fault reporting, street lighting, PURC regulation, contact directory, recent news

7.3 Live Scraper (`ecg_scraper.py`)

Function	Purpose
<code>scrape_outages()</code>	Main entry point — scrapes homepage, status index, and all 8 regional pages; deduplicates by title
<code>_fetch(url)</code>	HTTP GET with 20-second timeout and custom User-Agent; returns BeautifulSoup or None on failure
<code>_classify(text)</code>	Returns (type, status): <i>scheduled/active</i> for planned maintenance, <i>unplanned/active</i> otherwise
<code>_extract_cause(text)</code>	Pattern-matches title text to identify cause (broken HT pole, transformer fault, GRIDCo upstream issue, etc.)
<code>_normalise_title(text)</code>	Converts ALL-CAPS ECG article titles to readable Title Case; preserves ECG, HT, KV, GRIDCO, VRA, PURC acronyms
<code>_parse_iso_date(text)</code>	Extracts a date from title text (e.g. "16th June 2026") and returns ISO 8601 format

Scraping runs as an `asyncio` background task using `asyncio.to_thread()` to keep the synchronous BeautifulSoup work off the async event loop. A second background coroutine loops every 3600 seconds (one hour) to keep outage data fresh without requiring a server restart. Administrators can also trigger a manual re-scrape via `POST /admin/scrape`.

7.4 Database Schema (`database.py`)

Table	Key Columns	Purpose
<code>messages</code>	session_id, sender, content, intent, timestamp	Full chat log (user and bot)
<code>message_feedback</code>	message_id, rating (helpful/unhelpful)	Per-message thumbs up/down ratings
<code>chat_ratings</code>	session_id, stars (1–5), comment	Session-level star ratings with optional comments
<code>fault_reports</code>	reference, customer_name, phone, location, fault_type, urgency	Submitted electrical fault reports
<code>outages</code>	area, type, start_time, estimated_restoration, status, cause	Manually seeded outage data
<code>ecg_outages</code>	area, type, status, cause, source_title, scraped_at	Live ECG website scraped outage data (replaced each scrape)
<code>ecg_scrape_log</code>	scraped_at, total_outages, regions, success	Scrape history and audit trail
<code>escalations</code>	session_id, customer_name, intent, last_message, status	Human agent queue log
<code>rag_references</code>	message_id, session_id, sources (JSON)	Audit trail of which KB documents influenced each bot reply

7.5 API Endpoint Reference

Method	Path	Auth	Rate Limit	Description
GET	/	—	—	Health check
GET	/outages	—	—	All outages (seeded + live ECG data)
POST	/fault-report	—	5/min	Submit an electrical fault report
POST	/chat-rating	—	10/min	Submit a star rating for a session
POST	/message-feedback	—	30/min	Submit helpful/unhelpful on a message
WS	/ws/{session_id}	—	20 msg/min per session	Real-time chat connection
POST	/admin/login	—	—	Authenticate and receive session token
POST	/admin/logout	Token	—	Revoke the current session token
GET	/admin/fault-reports	Token	—	View submitted fault report queue
GET	/admin/escalations	Token	—	View human escalation list
GET	/admin/analytics	Token	—	Chatbot performance metrics
GET	/admin/rag-sources	Token	—	Knowledge base retrieval audit log
POST	/admin/scrape	Token	—	Manually trigger ECG website scrape
GET	/admin/scrape-log	Token	—	View scrape history
GET	/admin/ecg-outages	Token	—	View raw live outage data

8. Frontend Deep-Dive

8.1 Application Shell (`App.jsx`)

A single-page application with 3 customer-facing tabs (Chat Support, Outage Status, Report a Fault) and a route-based admin panel. The admin panel loads when the URL path begins with `/admin`, detected via `window.location.pathname` without needing a router library. A persistent header displays ECG branding, an Admin link, and a direct call link to 0800-POWER.

8.2 Chat Window (`ChatWindow.jsx`)

- **Exponential backoff reconnect:** If the WebSocket drops, the client retries after 1 second, then 2s, 4s, 8s, up to a maximum of 30 seconds. A "Reconnecting..." indicator shows during this period.
- **Typing indicator:** Three animated dots appear while waiting for the bot response, consistent with standard messaging conventions.
- **Quick reply chips:** The bot response includes suggested follow-up questions that render as clickable buttons below the conversation.
- **ISO date formatting:** Machine timestamps in bot responses (e.g. `2026-06-16T08:00:00`) are automatically converted to human-readable format (*Tue, 16 Jun 2026 at 08:00*).
- **Star rating prompt:** After the user receives 5 bot responses, a 1–5 star rating widget appears with an optional comment field, posting to `POST /api/chat-rating`.
- **Per-message feedback:** Every bot message displays thumbs up/down buttons that post *helpful/unhelpful* feedback to the backend using the message's unique database ID.
- **Human escalation card:** When the bot flags `escalate: true`, a highlighted banner showing the emergency number appears inline in the chat.

8.3 Admin Panel (`AdminPanel.jsx`)

Tab	What It Shows
Analytics	Average satisfaction score, response accuracy %, escalation rate %, intent distribution bar chart, satisfaction histogram (1–5 stars), recent user comments with timestamps
Fault Reports	Submitted fault reports with unique reference numbers, urgency colour-coding (critical/high/medium/low), customer name, phone, location, and description
Escalations	Sessions routed to human agents — customer name, intent category, last message, timestamp, and status (pending/resolved)
RAG Sources	Audit trail showing which knowledge base documents were retrieved for each bot response, linked by message ID and session

8.4 Network Configuration

All API and WebSocket URLs are proxy-relative, defined in a central `config.js` file. `API = '/api'` is proxied by Vite to the local backend. `WS_URL` uses `window.location.host`, so it dynamically resolves to whichever hostname or IP address the browser used to load the page — this is what allows the application to work on other devices without any configuration change.

9. Security Measures

Threat	Measure Implemented
Admin password exposed in browser JavaScript	Password removed from the frontend entirely. Now only sent once to <code>POST /admin/login</code> . Backend issues a 64-character cryptographic token (<code>secrets.token_hex(32)</code>); all subsequent admin calls use this opaque token.
Cross-Site Scripting (XSS) via AI-generated HTML	<code>DOMPurify.sanitize()</code> applied to every bot message before <code>dangerouslySetInnerHTML</code> rendering. Only safe tags are permitted: <code>strong</code> , <code>em</code> , <code>br</code> , <code>ul</code> , <code>li</code> , <code>p</code> .
REST API abuse and brute-force	<code>slowapi</code> rate limiter on all public mutation endpoints: 5/min on fault reports, 10/min on chat ratings, 30/min on message feedback, keyed by client IP address.
Chat flooding via WebSocket	Per-session WebSocket rate limit: maximum 20 messages per 60-second rolling window. Excess messages receive a polite throttle notice.
Secrets committed to source control	<code>.gitignore</code> explicitly excludes <code>backend/.env</code> , all <code>*.env</code> files, <code>backend/chatbot.db</code> , <code>frontend/node_modules/</code> , and <code>.venv/</code> .
SQL injection	All database queries use parameterised statements with the <code>sqlite3</code> placeholder syntax (<code>?</code>). No SQL string interpolation exists anywhere in <code>database.py</code> .
Unauthenticated access to admin data	All 7 admin endpoints protected by a FastAPI <code>require_admin</code> dependency that checks the Bearer token against the in-memory token set. Returns HTTP 401 if missing or invalid.

10. Changes and Improvements Made

After the initial working implementation was delivered, a systematic review identified 9 issues across security, reliability, data quality, and user experience. All 9 were fully resolved.

Change 1 — Backend Token Authentication

Problem: The admin password was hardcoded in the React frontend (`const ADMIN_PASSWORD = 'kanea2026'`). Any user who opened browser developer tools could read it from the source bundle.

Solution: Password moved to `backend/.env`. A new `POST /admin/login` endpoint validates the password and returns a cryptographic session token. All 7 admin endpoints now require a valid Bearer token enforced by a FastAPI dependency.

Change 2 — REST API Rate Limiting

Problem: Public-facing endpoints (fault-report, ratings) had no submission limits, leaving them open to automated abuse.

Solution: Added `slowapi` with per-endpoint IP-based limits. Fault reports: 5/min. Chat ratings: 10/min. Message feedback: 30/min.

Change 3 — Hourly Scraper Refresh Loop

Problem: The ECG website scraper only ran once at server startup. Outage data became stale throughout the day.

Solution: Added a `_scrape_loop()` asyncio coroutine that refreshes the outage data every hour indefinitely. Both the initial scrape and the loop run as background tasks without blocking the request handler.

Change 4 — Session Persistence After Restart

Problem: Conversation history was only in a Python dictionary. A server restart erased all session context, causing the chatbot to lose continuity with active users.

Solution: Added `get_session_history_for_chat()`. When a session is not found in memory, the chatbot reconstructs the last 4 turns from the `messages` table before the first LLM call after a restart.

Change 5 — TF-IDF RAG Scoring

Problem: The original RAG system counted keyword overlaps with equal weight. Common words dominated scores and caused irrelevant documents to rank highly.

Solution: Replaced keyword counting with TF-IDF scoring, pre-computed at module load time. Rare domain terms like PURC, prepaid, reconnection fee now score proportionally higher, producing more accurate document retrieval.

Change 6 — ECG Title Normalisation in Scraper

Problem: ECG article titles are often entirely in ALL-CAPS, reducing readability in the chatbot interface.

Solution: Added `_normalise_title()` which detects fully-uppercase titles and converts them to Title Case, while preserving known acronyms (ECG, HT, KV, GRIDCO, VRA, PURC).

Change 7 — DOMPurify XSS Sanitisation

Problem: Bot responses were rendered using `dangerouslySetInnerHTML` with no sanitisation, allowing any HTML injected via the LLM to execute in the user's browser.

Solution: All bot text passes through `DOMPurify.sanitize()` with a strict allowlist before rendering. A custom date formatter also converts machine timestamps to human-readable strings.

Change 8 — WebSocket Reconnect and Feedback UI

Problem: A dropped WebSocket silently broke the chat with no user feedback. The analytics tables (`chat_ratings` , `message_feedback`) had no UI, so the Analytics tab always showed "No data".

Solution: Added exponential backoff reconnect logic. Added a `StarRating` component (after the 5th bot response) and per-message thumbs-up/down buttons wired to the respective API endpoints.

Change 9 — .gitignore and LAN Network Access

Problem: No `.gitignore` existed (risking secret leakage). All URLs were hardcoded to `localhost` , preventing access from any other device.

Solution: Created a comprehensive `.gitignore` . Replaced all hardcoded URLs with proxy-relative paths via a central `config.js` . Enabled `host: true` in Vite and configured both servers to bind to `0.0.0.0` .

11. How to Run the System

Prerequisites

- Python 3.11 or higher with `pip`
- Node.js 18 or higher with `npm`
- A free Groq API key (obtainable from console.groq.com)

First-time Setup

1. Create and activate a Python virtual environment:
`python -m venv .venv` then `.venv\Scripts\activate` (Windows)
2. Install backend dependencies:
`cd backend && pip install -r requirements.txt`
3. Ensure `backend/.env` exists and contains:
`GROQ_API_KEY=your_key_here`
`ADMIN_PASSWORD=kanea2026`
4. Install frontend dependencies:
`cd frontend && npm install`

Starting the Application

Terminal 1 — Backend	Terminal 2 — Frontend
<code>cd backend</code>	<code>cd frontend</code>
<code>uvicorn main:app --host 0.0.0.0 --port 8000 --reload</code>	<code>npm run dev</code>

Access URLs

URL	Purpose
<code>http://localhost:5173</code>	Customer-facing chat application
<code>http://<LAN-IP>:5173</code>	Same app from any device on the same Wi-Fi network
<code>http://localhost:5173/admin</code>	Admin operations dashboard (password: <code>kanea2026</code>)
<code>http://localhost:8000/docs</code>	FastAPI auto-generated interactive API documentation

12. Known Limitations

Current Limitations

1. **No HTTPS — not production-ready for public access.** The system runs over plain HTTP. HTTPS is required for any public-facing deployment; browsers increasingly restrict plain WebSockets on non-HTTPS pages.
2. **Admin tokens lost on server restart.** Tokens are stored in a Python `set` in memory. Every backend restart forces all admins to sign in again.
3. **Seeded outage data is static and stale.** The 5 sample rows in the `outages` table have hardcoded dates from early 2026 and are now out of date. They should be removed or replaced with a managed demo dataset.
4. **Database path is relative to the working directory.** If uvicorn is started from a different folder, a new empty database is created rather than opening the existing one.
5. **RAG Sources tab label bug.** The Admin Panel's RAG Sources tab displays "Slide {source.id}" for all knowledge base entries, including those sourced from `ecg_knowledge.txt`. The label needs to branch on source type.
6. **No customer identity.** Users are anonymous (session ID only). There is no way to link chat sessions to a specific ECG account number or meter.

13. Recommended Future Improvements

Priority 1 — Production Deployment

- Deploy the FastAPI backend to a cloud VM (AWS EC2, DigitalOcean Droplet, or Render.com)
- Build the React frontend to static files (`npm run build`) and serve them via FastAPI `StaticFiles` or a CDN — this eliminates the Vite dev server in production
- Configure Nginx as a reverse proxy with a Let's Encrypt SSL certificate to enable HTTPS and WSS (secure WebSocket), which are required for any public deployment

Priority 2 — Data Quality

- Remove or replace the stale seeded outage rows in the `outages` table
- Expand `ecg_knowledge.txt` with tariff schedules, district office contact numbers, and updated connection application procedures
- Schedule a daily crawl of ECG news and announcements to automatically update the knowledge base

Priority 3 — New Features

- **Customer account lookup:** Allow customers to enter their meter number and retrieve billing balance, prepaid credit, or account status from an ECG data feed
- **Outage push notifications:** Allow customers to subscribe to notifications (SMS or email) for outages and restorations in their area
- **Fault report status tracking:** Let customers query the progress of their previously submitted fault report using its reference number (e.g. FAULT-A3B1C2)
- **Multi-language support:** Add Twi language support given ECG's primary customer base; the Groq LLM natively supports prompting in Twi
- **Live agent handoff:** Integrate with WhatsApp Business API or Twilio so that the escalation path actually connects to a real human agent, rather than displaying a phone number

Priority 4 — Engineering Quality

- Replace the in-memory admin token set with a database-backed store including expiry timestamps
- Move the database file path to an environment variable so it is configurable without changing code

- Add automated tests: pytest for the backend AI pipeline and API endpoints; Playwright or Cypress for the frontend chat flow
- Replace `print()` statements with structured JSON logging and proper log levels (INFO, WARNING, ERROR) to enable log aggregation in production
- Fix the RAG Sources tab label in AdminPanel to correctly identify `ecg_website` vs. `presentation` sourced knowledge base entries

14. Conclusion

The Kanea AI Customer Support System delivers a complete, working Phase 2 implementation against all seven in-scope objectives. The chatbot demonstrates intelligent query routing, distinguishing live operational questions from static FAQ queries. It integrates real-time data directly from the ECG website, provides a secure admin operations dashboard, and collects measurable user satisfaction data through an in-chat rating system.

The system is currently deployed on a local network and is accessible to any device on the same Wi-Fi. It is ready for a supervised pilot test with ECG staff or a small group of customers. The primary remaining step before public release is production infrastructure: HTTPS termination via a reverse proxy and cloud hosting.

The architecture is intentionally lean and maintainable: no Docker, no external message broker, no complex dependency chain. The backend deploys to any Python hosting environment; the frontend builds to static files. This design lowers the barrier for handover and ongoing maintenance by non-specialist ECG IT staff.

Project Summary

- 8 Python packages
- 5 React components
- 9 database tables
- 15 API endpoints
- 14 ECG knowledge sections
- 8 regional scrapers
- 3-step AI pipeline
- 9 security improvements
- HTTPS deployment pending